

SkillMatch AI

Comprehensive Work & Data Flow Architecture Specification Student & Employer Portals, State Transitions, and UI/UX Revamp Guide

Author: Lead AI & Mobile Systems Engineer | Target: Mobile UI Engineering Team | Status: Complete Architecture Spec

1. Executive Summary & Platform Topology

SkillMatch AI is an intelligent internship and early-career job placement ecosystem designed to eliminate recruitment friction for students and employers. The platform bridges the gap between unstructured candidate profiles (resumes) and job specifications using a hybrid matching formula combining ontology-driven skills taxonomy normalization (0.6 weight) and 384-dimensional dense semantic vector embeddings (0.4 weight).

CORE ECOSYSTEM VALUE PILLARS

- **Student Experience:** Frictionless resume upload (PDF/DOCX), automatic skill extraction, human-in-the-loop verification, hybrid-scored opportunity discovery feed, and live recruitment status tracking.
- **Employer Experience:** Rapid job publishing, taxonomy skill auto-tagging, real-time AI-ranked applicant pool review, candidate explainability breakdowns, and 1-click status progression with automated student notifications.
- **Data Privacy & Compliance:** Full compliance with Republic Act No. 10173 (Philippine Data Privacy Act of 2012) supporting machine-readable data export (/auth/me/export) and permanent account/file erasure (/auth/me).

2. Student Portal: End-to-End Work & Data Flow

The Student Portal guides candidate onboarding from resume ingestion to job matching and interview tracking. Below is the sequential breakdown across screens, UI interactions, API payloads, and data transformations.

Phase 2.1: Authentication & Session Initialization

- **Screen / Component:** LoginScreen.tsx / RegisterScreen.tsx
- **User Workflow:** Student inputs Full Name, Email, Password, and selects role = 'student'. On submission, app authenticates and persists JWT token securely.
- **API Endpoint:** POST /api/v1/auth/register or POST /api/v1/auth/login
- **Outgoing Payload:**

```
{ "email": "student@univ.edu", "password": "...", "full_name": "Juan Dela Cruz", "role": "student" }
```
- **Response & State Mutation:** Returns JWT access_token + User object. AuthContext stores token in Expo SecureStore and sets Axios/Fetch default authorization header Bearer <token>.

Phase 2.2: Resume Ingestion, Extraction & Embedding Generation

- **Screen / Component:** ResumeUploadScreen.tsx
- **User Workflow:** Student taps 'Select Document' -> DocumentPicker opens native file selector for PDF or DOCX -> File size (<5MB) verified -> Student taps 'Upload & Parse Resume' -> Animated scanning state renders while backend processes.
- **API Endpoint:** POST /api/v1/resumes/upload (Content-Type: multipart/form-data)
- **Backend Data Transformation Pipeline:**
 1. Magic Byte Validation: Validates true MIME signatures (puremagic) to prevent renamed malicious payloads.
 2. Isolated Storage: File saved to storage/resumes/{user_id}/{uuid}.pdf with sanitized paths.
 3. Text Extraction: PDFPlumber / python-docx extracts raw unstructured textual content.
 4. LLM Field Extractor: OpenAI/Claude (with heuristic fallback) extracts structured JSON containing skills, education, experience, certifications, summary.
 5. Taxonomy Normalization: Skills mapped to canonical dictionary (e.g. 'React.js' -> 'React', 'py' -> 'Python').
 6. 384-Dim Vector Embedding: Sentence-Transformers generates dense semantic vector embedding from synthesized resume text representation.
- **Response:**

```
{ "id": "res_123", "status": "parsed", "parsed_data": { "skills": ["Python", "FastAPI", "React", "PostgreSQL"], "education": [...], "experience": [...] } }
```

Phase 2.3: Human-in-the-Loop Skill Verification & Editing

- **Screen / Component:** ResumeReviewScreen.tsx
- **User Workflow:** Student reviews extracted skills rendered as dismissible chips. Student can remove inaccurate tags, add missing skills via text input, and tap 'Save & Activate Profile'.
- **API Endpoint:** PUT /api/v1/resumes/{resume_id}/parsed-data
- **Outgoing Payload:**

```
{ "parsed_data": { "skills": ["Python", "Docker", "FastAPI", "PostgreSQL"], ... }, "status": "active" }
```
- **Backend Action:** Normalizes updated skills list, regenerates the 384-dimensional vector embedding, updates ResumeSkill relational records, and marks status as 'active'.

Phase 2.4: Recommendation Discovery & Opportunity Matching Feed

- **Screen / Component:** RecommendationFeedScreen.tsx (Main Tab: 'Opportunities')
- **User Workflow:** Student browses AI-matched job opportunities. Feed merges Internal Employer Postings with External Synced Jobs (JSearch/RapidAPI). Each card displays match percentage badge, company name, job type chip (Internship / OJT / Full-time), location, and explainability breakdown (matched skills in green, missing skills in amber).
- **API Endpoint:** GET /api/v1/matches/recommendations
- **Hybrid Scoring Formula (Backend):**
$$\text{Final Match Score} = (0.60 * \text{Taxonomy Skill Overlap Score}) + (0.40 * \text{Cosine Similarity}(\text{Resume Vector}, \text{Job Vector}))$$
- **Response Item Schema:**

```
{  "match_score": 0.88,  "skill_score": 0.85,  "target_type": "internal",
```

```

    "target": { "id": "job_01", "title": "Junior Python / Backend Developer", "employer": { "company_name": "Tech Corp"
  } },
  "explanation": {
    "matched_skills": ["Python", "FastAPI", "PostgreSQL"],
    "missing_skills": ["Docker", "Redis"],
    "summary": "You possess 3 of 5 required skills (88% overall alignment)."
  }
}

```

Phase 2.5: Application Submission & Live Status Stepper

- **Screen / Component:** RecommendationFeedScreen.tsx (Action) -> ApplicationTrackerScreen.tsx (View)
- **User Workflow:** Student taps 'Apply Now' on an internal job card -> Optional cover note modal opens -> On confirm, application record is created -> Student navigates to 'Applications' tab to see visual 5-step progress stepper.
- **API Endpoints:**
 - POST /api/v1/applications/ (Payload: { "job_posting_id": "job_01", "notes": "Eager to contribute." })
 - GET /api/v1/applications/my-applications
- **5-Step Visual Stepper States:** 1. Submitted → 2. Under Review → 3. Shortlisted → 4. Interview Scheduled → 5. Accepted (or Rejected/Withdrawn).

Phase 2.6: In-App Status Notification Alerts

- **Component / Endpoint:** GET /api/v1/notifications/ and GET /api/v1/notifications/unread-count
- **Event Trigger:** When an employer updates candidate application status, the backend dispatches a structured in-app notification to the student with notification_type = 'application_status_update'.

3. Employer Portal: End-to-End Work & Data Flow

The Employer Portal empowers corporate recruiters and hiring managers to publish vacancies, automatically ingest skill requirements, and review candidate applicants pre-sorted by AI match affinity.

Phase 3.1: Employer Registration & Company Profile

- **Screen / Component:** RegisterScreen.tsx (Role: 'employer')
- **User Workflow:** Employer signs up with Company Name, Work Email, and Password. Backend atomically creates User record and linked EmployerProfile record.
- **API Endpoint:** POST /api/v1/auth/register (role: 'employer')

Phase 3.2: Job Posting Creation & Vector Indexing

- **Screen / Component:** EmployerPostingsScreen.tsx (Modal: 'Create New Job Posting')
- **User Workflow:** Recruiter taps '+ Create Job Posting' -> Fills Title, Job Type ('internship', 'ojt', 'full_time', 'part_time'), Location, Remote/On-site toggle, Required Skills (comma-separated), Preferred Skills, and Description -> Taps 'Publish Posting'.

- **API Endpoint:** POST /api/v1/jobs/

- Outgoing Payload:

```
{
  "title": "Backend Engineering Intern",
  "description": "Looking for passionate Python/FastAPI interns...",
  "job_type": "internship",
  "location": "Makati / Remote",
  "required_skills": ["Python", "FastAPI", "PostgreSQL", "Git"],
  "preferred_skills": ["Docker", "Pytest"]
}
```

- **Backend Action:** Skills normalized via ontology; 384-dimensional dense vector embedding generated from synthesized job text representation; posting stored with status = 'active'.

Phase 3.3: Active Postings Dashboard

- **Screen / Component:** EmployerPostingsScreen.tsx
- **User Workflow:** Recruiter views list of all active company postings with applicant count badges (e.g. '12 Applicants'). Recruiter taps 'View Ranked Applicants' on a card to navigate into the candidate review pipeline.
- **API Endpoint:** GET /api/v1/jobs/my-postings

Phase 3.4: AI-Ranked Candidate Applicant Review

- **Screen / Component:** EmployerApplicantsScreen.tsx
- **User Workflow:** Recruiter views all applicants for the selected posting. Candidates are automatically pre-sorted descending by hybrid AI Match Score (e.g., #1 Juan Dela Cruz - 94% Match, #2 Maria Santos - 82% Match). Each card displays matched skill badges, missing skill alerts, applicant notes, current recruitment status badge, and 'Download Resume' button.
- **API Endpoint:** GET /api/v1/applications/posting/{posting_id}/applicants
- Ranked Applicant Object Schema:

```
{
  "application_id": "app_55",
  "candidate_name": "Juan Dela Cruz",
  "candidate_email": "juan@univ.edu",
  "status": "under_review",
  "match_score": 0.94,
  "skill_score": 0.90,
  "resume_url": "/api/v1/resumes/files/user_01/resume_uuid.pdf",
  "explanation": {
    "matched_skills": ["Python", "FastAPI", "PostgreSQL", "Git"],
    "missing_skills": ["Docker"],
    "summary": "Candidate matches 4 of 5 required skills with strong semantic vector alignment."
  }
}
```

Phase 3.5: Recruitment Pipeline Status Progression & Notifications

- **Screen / Component:** EmployerApplicantsScreen.tsx (Action Buttons on candidate card)
- **User Workflow:** Recruiter taps 1-click status progression buttons: 'Shortlist', 'Schedule Interview', 'Accept Candidate', or 'Reject'.
- **API Endpoint:** PATCH /api/v1/applications/{application_id}/status
- **Outgoing Payload:** { "status": "interview_scheduled" }
- **Automated Backend Trigger:** Backend updates Application record AND immediately dispatches an in-app notification to the student's account ('Interview Scheduled: Tech Corp has scheduled an interview for Backend Engineering Intern.').

4. Master API & Data Flow Matrix

Summary of all API routes, HTTP methods, authorization roles, and primary database models:

Module	HTTP & Route	Auth / Role	Primary Input / Payload	Output & Side Effects
Auth	POST /auth/register	Public	email, password, full_name, role	Creates User (+ EmployerProfile if employer), returns JWT
Auth	POST /auth/login	Public	username (email), password	Returns JWT access_token + user data
Auth	GET /auth/me/export	Student / Employer	None (Bearer JWT)	RA 10173 complete JSON personal data export
Auth	DELETE /auth/me	Student / Employer	None (Bearer JWT)	RA 10173 permanent cascading delete + disk purge
Resumes	POST /resumes/upload	Student	Multipart PDF/DOCX file	Parses raw text, LLM JSON extract, vector embedding
Resumes	PUT /resumes/{id}/parsed-data	Student	parsed_data JSON	Human verification: re-normalizes skills & vector
Resumes	DELETE /resumes/{id}	Student	None	Deletes resume record

and physical file on disk

Jobs	POST /jobs/	Employer	title, job_type, skills, desc	Creates internal posting, builds 384-dim job embedding
Jobs	GET /jobs/my-postings	Employer	None	Returns employer's active postings with applicant counts
Matches	GET /matches/recommendations	Student	None	Hybrid scored feed of internal + external jobs
Applications	POST /applications/	Student	job_posting_id, notes	Creates application in 'submitted' status
Applications	GET /applications/my-applications	Student	None	Returns student's applications with 5-step status
Applications	GET /applications/posting/{id}/applicants	Employer	None	Returns AI-ranked applicant pool descending
Applications	PATCH /applications/{id}/status	Employer	status enum	Transitions status & dispatches student notification
Notifications	GET /notifications/	Authenticated	None	Returns user's in-app notification activity feed
Evaluation	POST /evaluation/run-benchmark	Coordinator/Admin	None	Runs Precision@K, NDCG@K, MRR benchmark

5. State Machines & Status Enums

5.1 Application Status Lifecycle

The recruitment lifecycle follows a strict sequential state progression with terminal states:

- submitted: Initial state upon student application.
- under_review: Recruiter has opened and viewed the candidate profile.
- shortlisted: Candidate marked as high-potential match for interview pool.
- interview_scheduled: Candidate invited for technical/behavioral interview.
- accepted (Terminal Success): Offer extended and accepted.
- rejected (Terminal): Application declined.
- withdrawn (Terminal): Student voluntarily withdrew application.

5.2 Resume Parsing Status Lifecycle

• uploaded → parsing → parsed (Awaiting student verification) → active (Verified & matched) → archived

6. UI/UX Improvement Blueprint & Redesign Checklist

To elevate the mobile experience from functional prototype to an industry-grade, visually stunning flagship product, the following concrete UI/UX improvements are recommended for implementation across the mobile app codebase:

Design Tokens & Theme System

- Replace hardcoded hex colors with a centralized Theme / Token system (src/theme/colors.ts).
- Implement a cohesive Dark Glassmorphism palette: Background (#090D16), Surface (#111827), Elevated Surface (#1F2937), Primary Indigo (#6366F1), Emerald Success (#10B981), Amber Warning (#F59E0B), Rose Danger (#EF4444).
- Typography hierarchy: Load Google Fonts (e.g. Outfit or Inter) with dedicated display styles for headings, badges, and metadata labels.

Student Screen Improvements

- ResumeUploadScreen: Add an interactive animated upload drop zone with file type icons, real-time upload progress bar, and pulsating AI parsing skeleton loader.
- ResumeReviewScreen: Add categorized skill pills (Languages, Frameworks, Cloud, Databases) with a quick-tag search bar for seamless skill additions.
- RecommendationFeedScreen: Implement interactive swipeable job cards with a circular gradient Match Score Gauge (e.g., 94%), match breakdown accordion, and sticky Filter/Search bar.
- ApplicationTrackerScreen: Replace text status with an animated vertical/horizontal recruitment timeline stepper with pulsing active nodes and status timestamps.

Employer Screen Improvements

- EmployerPostingsScreen: Redesign job cards with mini-applicant avatars, quick-toggle active/pause switches, and a streamlined multi-step 'Post a Job' modal with skill tag autocompletion.
- EmployerApplicantsScreen: Add sorting/filtering toolbar (Sort by: Highest AI Match, Most Recent, Status) and an expandable candidate dossier view displaying full resume text and skill match comparison side-by-side.

Micro-Interactions, Feedback & Empty States

- Skeleton Loaders: Replace generic ActivityIndicator spinners with shimmer skeleton placeholders matching card layouts.

- Empty States: Design custom SVG/vector illustrations with contextual action buttons for 'No Recommendations Yet', 'No Applications Yet', and 'No Applicants Yet'.
- Haptic Feedback: Add subtle Expo Haptics (`Haptics.selectionAsync()`, `Haptics.notificationAsync()`) on button taps, status updates, and successful uploads.

Offline Resilience & Error Handling

- Toast Notification Banner: Add a floating toast banner for network connection alerts, API error messages, and action confirmations.
- Pull-to-Refresh: Implement `RefreshControl` across all feed and list screens with smooth indicator animations.